

RAG Masterclass 4-Hour Study Guide

RAG Masterclass — Your 4-Hour Study Guide

Built from YOUR diary notes (Oct 19 → Oct 24) · Interview-first · Hands-on in Jupyter

How to use this: Each hour = Read (25 min) → Code in Jupyter (25 min) → Interview drill (10 min). Every concept below appears in your handwritten notes. I've expanded each one the way the professor *meant* to explain it — slowly, with analogies.

🕒 HOUR 1 — Basic RAG: The Foundation

(Your notes: Oct 19 — "RAG Pipelines (Basic RAG)", Text Content → Embedding, Vector Search, Cosine Similarity, Chunks)

1.1 Why does RAG even exist?

An LLM (like GPT or Claude) is like a very smart student who finished studying **months ago** and then walked into the exam hall. It knows a lot, but:

- It doesn't know **your company's documents** (your invoices, policies, contracts).
- It doesn't know anything **after its training date**.
- If you ask it something it doesn't know, it may **confidently make things up** (hallucinate).

RAG = Retrieval-Augmented Generation = "Before answering, go look it up in the books, THEN answer."

It's an open-book exam instead of a closed-book exam. That's the whole idea.

1.2 The Basic RAG pipeline (your Oct 19 diagram, decoded)

Your notes show two parallel tracks — this is the key picture:

INDEXING (done once, offline)	QUERY TIME (every user question)
All data → Chunks Chunks → Embeddings Store in Vector DB	Search Query ("What is the amount in invoice ABC23?") Query → Embedding Vector Search (compare query embedding vs chunk embeddings) Top chunks → give to LLM → Answer

Let's unpack each word:

Chunking — You can't embed a 200-page PDF as one blob. You cut it into bite-sized pieces (chunks), typically 200–1000 tokens each. *Analogy: You don't swallow a whole roti — you tear it into pieces.* Why it matters: too big → chunk contains noise; too small → chunk loses context. This is a classic interview question.

Embedding — A model (e.g., all-MiniLM-L6-v2, OpenAI's text-embedding-3) converts text into a **list of numbers (a vector)**, e.g., 384 or 1536 numbers. The magic: **texts with similar MEANING get vectors that point in similar directions.**

- "How much does the invoice cost?" and "invoice amount" → vectors close together
- "invoice amount" and "cricket score" → vectors far apart

Analogy: An embedding is like GPS coordinates for meaning. Delhi and Gurgaon have nearby coordinates; Delhi and Tokyo don't. Embeddings give every sentence a location in "meaning-space."

Cosine Similarity — Your notes say $C1 = 0.9$. This is how we measure "how close are two vectors?" It measures the **angle** between them:

- 1.0 → same direction → same meaning
- 0.0 → unrelated
- 0.9 (your note!) → very similar → strong match

Vector Search — Take the query's embedding, compare it (cosine similarity) against every chunk's embedding, return the chunks with the highest scores. That's it. That's vector search.

1.3 Your professor's example (bottom of Oct 19 page)

Q: "What is the amount in invoice ABC23?"

He split this into two arrows: **Vector (Nearest)** and **Keyword** — because this query exposes vector search's weakness:

- **Vector search** understands "amount" $\approx$ "total" $\approx$ "sum payable" (meaning). ✓
- But "ABC23" is an exact ID. Embedding models are bad at exact codes, part numbers, names. Vector search might return invoice ABC29's chunk because it "feels similar." ✗
- **Keyword search** (exact word matching) nails "ABC23" instantly. ✓

☞ This is *why* the class moved to **Hybrid Search** next. Remember this example — it's a perfect interview answer for "When does vector search fail?"

1.4 🖱 Hands-on (Jupyter) — 25 min

```
!pip install sentence-transformers -q

from sentence_transformers import SentenceTransformer, util

model = SentenceTransformer('all-MiniLM-L6-v2')

# Our tiny "document store" — pretend these are chunks from PDFs
chunks = [
    "Invoice ABC23 has a total amount of Rs. 45,000 due on March 5.",
    "Invoice ABC29 total payable is Rs. 12,500.",
    "Climate change increases global temperatures and sea levels.",
    "The refund policy allows returns within 30 days of purchase.",
]

# 1. INDEXING: chunks -> embeddings
chunk_embeddings = model.encode(chunks)
print("Each chunk became a vector of size:", chunk_embeddings.shape) # (4, 384)

# 2. QUERY TIME
query = "What is the amount in invoice ABC23?"
query_embedding = model.encode(query)

# 3. Cosine similarity against every chunk
scores = util.cos_sim(query_embedding, chunk_embeddings)[0]
for chunk, score in sorted(zip(chunks, scores), key=lambda x: -x[1]):
    print(f"{{score:.3f}} | {{chunk[:60]}}")
```

Try this experiment: change the query to just "ABC23" and watch the scores — notice how vector search struggles with pure IDs. You just *proved* the professor's point yourself.

1.5 🧩 Interview drill (Hour 1)

1. **What is RAG and why do we need it?** → LLMs have frozen knowledge + hallucinate; RAG retrieves relevant documents at query time and grounds the answer in them.
2. **Walk me through a basic RAG pipeline.** → Chunk → Embed → Store in vector DB || Query → Embed → Vector search (cosine sim) → Top-k chunks → LLM prompt → Answer.
3. **What is an embedding?** → Dense numeric vector representing meaning; similar meaning → similar vectors.
4. **What is cosine similarity?** → Angle-based similarity between vectors, ranges roughly 0→1 for text; higher = more similar.
5. **Why chunk documents?** → Embedding models have token limits + retrieval precision: you want to fetch the *relevant paragraph*, not a whole book.

🕒 HOUR 2 — Hybrid Search, RRF & the Cross Encoder

(Your notes: Oct 19 "Hybrid Search = Vector + Keyword (BM25) — Used in production", Oct 20 full page, Oct 22 "(query + Document) → BERT → 0.2, Cross Encoder very important")

This is the **heart of the masterclass** and the highest-value interview material. Your Oct 20 page is literally a production retrieval architecture.

2.1 Keyword Search & BM25

BM25 is the classic keyword-ranking algorithm (what search engines used for decades). It scores documents by:

- How many times the query words appear in the doc (term frequency)
- How *rare* those words are overall (rare words like "ABC23" score huge; common words like "the" score ~0)
- Normalized by document length

Analogy: BM25 is a librarian who matches your exact words. The vector model is a librarian who understands what you mean. One is literal, one is intuitive. Each fails where the other shines.

	Vector search	Keyword (BM25)
"amount" vs "total payable"	✔ understands	✗ misses
Exact IDs "ABC23"	✗ weak	✔ perfect
Typos/synonyms	✔	✗
Names, codes, jargon	✗	✔

Hybrid Search = run BOTH, combine the results. Your note "Used in production" is the professor telling you: real companies never rely on vector search alone.

2.2 The problem: two different ranked lists (your Oct 20 diagram)

Your notes:

Vector: [D1, D7, D9]	Keyword: [D7, D20, D81]
----------------------	-------------------------

Vector search says the best docs are D1, D7, D9. Keyword search says D7, D20, D81. **Different lists, different scoring scales** (cosine gives 0–1, BM25 gives 0–30+). You can't just add the scores. So how do you merge?

2.3 Reciprocal Rank Fusion (RRF)

(Your notes say "Reciprocal Range Fusion" — small correction: it's **Reciprocal RANK Fusion**. Fix this before your interview!)

RRF's genius: **ignore the scores, use only the RANKS (positions)**.

Formula: for each document, $RRF\ score = \sum \frac{1}{(k + rank)}$ across every list it appears in (k is a constant, usually 60; rank starts at 1).

Work through your professor's exact example (k=60):

Doc	Vector rank	Keyword rank	RRF score
D7	2	1	$\frac{1}{62} + \frac{1}{61} = \mathbf{0.0325}$ 🏆
D1	1	—	$\frac{1}{61} = 0.0164$ 🥉
D9	3	—	$\frac{1}{63} = 0.0159$ 🥉
D20	—	2	$\frac{1}{62} = 0.0161$

D7 wins because BOTH search methods liked it. That's exactly your notes' output: D7, D1, D9 → Reranked Documents . A document that appears in both lists gets boosted — agreement between two independent judges is strong evidence.

Analogy: Two judges rank dancers. Instead of comparing their scoring styles (one gives /10, one gives /100), you just look at podium positions. Anyone on BOTH podiums is clearly great.

2.4 The production pipeline (your Oct 20 "Coding Part" flowchart)

This is gold. Memorize this flow — it's a complete answer to "Design a production RAG retrieval system":

User Query
↓
Retrieve 100 chunks ← BM25 (keyword) + Vector Search [FAST, ROUGH]
↓
RRF (optional): merge the multiple ranked lists
↓
Top 20 chunks
↓
Cross Encoder: scores each (query, chunk) pair [SLOW, PRECISE]
↓
Top 5 chunks → LLM → Answer (your Oct 22 note: "Top 5 chunks")

Notice the funnel: **100 → 20 → 5**. Cheap methods do the rough cut; the expensive method polishes the finalists.

Analogy: Hiring. 100 resumes screened by keyword filter (fast). Top 20 get a phone screen. Top 5 get a full onsite interview (expensive but accurate). You'd never onsite-interview all 100.

2.5 Cross Encoder — "very important" (you underlined it!)

Two ways to compare a query and a document:

Bi-encoder (what vector search uses): embed query and doc *separately*, then compare vectors. Fast — doc embeddings are pre-computed. But the model never sees query and doc *together*, so it misses fine-grained interactions.

Cross-encoder (your Oct 22 note: (query + Document) → BERT → 0.2): concatenate query + document into ONE input, feed to a BERT-style model, it outputs a single relevance score (like 0.2 = not relevant, 0.9 = very relevant). Because the model reads them **together**, attention flows between every query word and every doc word → far more accurate.

The catch: you can't pre-compute anything. Every (query, chunk) pair is a fresh model run. Scoring 1 million chunks per query = impossible. Scoring 20 = easy. **That's why it sits at the END of the funnel.** And that's why your notes say "Used heavily in prod" — it's the accuracy booster every serious RAG system has.

Analogy: Bi-encoder = judging compatibility from two dating profiles read separately. Cross-encoder = watching the two people have an actual conversation. The conversation reveals far more — but you can't have conversations with a million people.

2.6 🧑 Hands-on (Jupyter) — 25 min

```
!pip install rank_bm25 sentence-transformers -q

from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer, CrossEncoder, util

chunks = [
    "Invoice ABC23 has a total amount of Rs. 45,000 due on March 5.",
    "Invoice ABC29 total payable is Rs. 12,500.",
    "Climate change increases global temperatures and sea levels.",
    "The refund policy allows returns within 30 days.",
    "Payment for ABC23 was processed via bank transfer.",
]
query = "What is the amount in invoice ABC23?"

# --- Track 1: BM25 keyword search ---
bm25 = BM25Okapi([c.lower().split() for c in chunks])
bm25_ranking = sorted(range(len(chunks)),
                      key=lambda i: -bm25.get_scores(query.lower().split())[i])

# --- Track 2: Vector search ---
model = SentenceTransformer('all-MiniLM-L6-v2')
sims = util.cos_sim(model.encode(query), model.encode(chunks))[0]
vec_ranking = sorted(range(len(chunks)), key=lambda i: -sims[i])

# --- RRF fusion (exactly your professor's D7/D1/D9 logic) ---
def rrf(rankings, k=60):
    scores = {}
    for ranking in rankings:
        for rank, doc_id in enumerate(ranking, start=1):
            scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)

fused = rrf([bm25_ranking, vec_ranking])
print("RRF order:", [chunks[i][:40] for i in fused])

# --- Cross Encoder reranking of the fused top results ---
ce = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
pairs = [(query, chunks[i]) for i in fused[:3]]
ce_scores = ce.predict(pairs) # one score per (query, chunk) pair!
for (q, c), s in sorted(zip(pairs, ce_scores), key=lambda x: -x[1]):
    print(f"{s:.3f} | {c[:50]}")
```

Run it and watch the funnel work: BM25 + vector disagree slightly → RRF merges → cross encoder gives the final precise ordering.

2.7 🧑 Interview drill (Hour 2) — these WILL be asked

1. **What is hybrid search and why use it?** → BM25 catches exact terms/IDs, vectors catch meaning; combining covers both failure modes; standard in production.
2. **What is RRF?** → Rank-based fusion: $\text{score} = \sum 1/(k+\text{rank})$; no score normalization needed; docs appearing in multiple lists get boosted.
3. **Bi-encoder vs cross-encoder?** → Bi: separate embeddings, fast, pre-computable, less accurate. Cross: joint input, slow, most accurate. Use bi for recall (100s), cross for precision (top 20).
4. **Why not run the cross encoder on all documents?** → $O(n)$ model inferences per query; latency and cost explode. Funnel: cheap-wide → expensive-narrow.
5. **Design a retrieval pipeline for production.** → Recite the Oct 20 flowchart: 100 (BM25+vector) → RRF → 20 → cross encoder → 5 → LLM.

🕒 HOUR 3 — Semantic Caching, Agentic RAG & Frameworks

(Your notes: Oct 22 "Long Chain → Framework", Oct 23 full page)

3.1 Quick fix on names (write these corrections in your diary!)

- **"Long Chain" = LangChain** — the most popular framework for building LLM/RAG pipelines (chains together loaders, splitters, retrievers, LLMs).
- **"Long Chain Graph" = LangGraph** — LangChain's sibling for building **agent workflows as graphs** (nodes = steps, edges = decisions, supports loops & branching). Saying "Long Chain" in an interview would hurt, so drill the real names: **LangChain**, **LangGraph**.

3.2 Semantic Caching — "Do we need to retrieve at all?" (your Oct 23 heading)

That question is the whole idea. In a real app, thousands of users ask **nearly the same questions**:

- "What is your refund policy?"
- "How do refunds work?"
- "Tell me about returns"

Why run the full pipeline (embed → search → rerank → LLM, costing money and 2–5 seconds) three times for what is the *same question*?

Semantic cache: store (query embedding → final answer) pairs. On a new query:

1. Embed the incoming query.
2. Compare against cached query embeddings (cosine similarity).
3. If similarity > threshold (e.g., 0.9 — remember your `c1 = 0.9` note?) → **return the cached answer instantly**. Cache HIT. 💰 saved.
4. Otherwise → run full RAG, then store the new pair. Cache MISS.

It's "semantic" because it matches by **meaning**, not exact text — a normal cache would treat "refund policy?" and "how do refunds work" as different keys.

Redis (your note: "you can use Redis cache") = an in-memory database, lightning fast, the standard tool for caches. Modern Redis even supports vector similarity search natively.

Analogy: A shopkeeper who's answered "when do you close?" 500 times stops thinking and instantly replies "9 PM" — even when phrased as "what time do you shut?" He matched the meaning, not the words. That's a semantic cache.

3.3 Agentic RAG — the Router LLM (your Oct 23 diagram)

Basic RAG is dumb in one way: it retrieves for **every** query, even "Hi, how are you?" (nothing to retrieve!) or "What's the weather right now?" (your documents can't know!).

Agentic RAG adds a brain at the front door. Your diagram:

```
Query → [ Router LLM ] → Web Search  (needs fresh/live info)
                        → RAG         (answerable from your documents)
                        → Answer      (LLM can answer directly, no retrieval)
```

A small/fast LLM first **classifies the query** and routes it to the right tool. The system now makes *decisions* — that's what makes it "agentic" (agent = something that decides and acts, not just executes a fixed pipe).

This connects beautifully to semantic caching — both answer the same meta-question: **"Is the expensive path actually necessary for THIS query?"**

LangGraph is exactly the tool for building this: Router = a node, each path = a branch, and you can loop (e.g., "answer not good enough → retrieve again").

3.4 🖱️ Hands-on (Jupyter) — 25 min

Build a mini semantic cache + a rule-based router (no API keys needed):

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer('all-MiniLM-L6-v2')

# ----- Semantic Cache -----
cache = [] # list of (embedding, answer)

def expensive_rag_pipeline(query):
    print("    ⚠️ running FULL RAG (slow, costly)...")
    return f"[Answer to: {query}]"
```

```

def ask(query, threshold=0.85):
    q_emb = model.encode(query)
    for emb, ans in cache:
        if util.cos_sim(q_emb, emb).item() > threshold:
            print("    ✨ CACHE HIT – instant, free!")
            return ans
    ans = expensive_rag_pipeline(query)
    cache.append((q_emb, ans))
    return ans

ask("What is your refund policy?")      # MISS → full pipeline
ask("How do refunds work?")            # HIT  → semantic match!
ask("What is the invoice amount?")     # MISS → different meaning

# ----- Router (Agentic RAG idea) -----
routes = {
    "web_search": "latest news current weather today stock price",
    "rag":         "invoice policy document contract report amount",
    "direct":      "hello hi thanks who are you joke",
}
route_embs = {k: model.encode(v) for k, v in routes.items()}

def route(query):
    q = model.encode(query)
    return max(routes, key=lambda k: util.cos_sim(q, route_embs[k]).item())

for q in ["What's the weather in Delhi today?",
          "What is the amount in invoice ABC23?",
          "Hi there!"]:
    print(f"{q[:45]} → {route(q)}")

```

You just built a baby semantic cache AND a baby router. In interviews, describing this tiny build shows *real* understanding.

3.5 🦾 Interview drill (Hour 3)

1. **What is semantic caching?** → Cache keyed by query *embedding*; similarity above a threshold returns the stored answer; cuts cost & latency for repeated/rephrased questions.
2. **What could go wrong with it?** → Threshold too low → wrong answers served for merely-similar queries; stale answers when documents update → need TTL/invalidation.
3. **What makes RAG "agentic"?** → An LLM makes routing/tool decisions (retrieve vs web search vs direct answer vs re-retrieve) instead of a fixed pipeline.
4. **LangChain vs LangGraph?** → LangChain: linear chains/components for LLM apps. LangGraph: graph-based agent workflows with branching, loops, and state.
5. **Where does Redis fit in a RAG system?** → In-memory store for the semantic cache (and sessions/rate-limiting); microsecond reads vs seconds for full RAG.

🦾 HOUR 4 — Scaling RAG to Millions of Users

(Your notes: Oct 22 "How to scale to Millions of Users? → Brute Force $O(n)$ → HNSW", Oct 23 "Scalability: AWS, Nginx load balancer, API Gateway" diagram, Oct 24 "Horizontal/Vertical scaling, Kubernetes")

Two separate scaling problems — don't mix them in an interview: **A) Search scaling** (millions of vectors) and **B) Traffic scaling** (millions of users).

4.1 (A) Scaling the SEARCH: Brute Force vs HNSW

Brute force (your note: "searching query one by one, $O(n)$ "): compare the query embedding with **every** stored vector. 10M chunks = 10M comparisons *per query*. Fine for a demo, dead in production.

HNSW — Hierarchical Navigable Small World (your note, spelled right — nice!). An *Approximate* Nearest Neighbor (ANN) index:

- Vectors are connected in a graph with **layers**: top layers have few nodes with long-range links (highways), bottom layers are dense with short links (streets).
- Search starts at the top, takes big jumps toward the target region, then descends layer by layer, refining locally.
- Complexity $\approx O(\log n)$ instead of $O(n)$. Tiny accuracy trade-off (~"approximate"), massive speedup.

Analogy: Finding a house in a new city. Brute force = knock on every door in the country. HNSW = fly to the right city (top layer), take a highway to the right district (middle), then walk the streets (bottom). You "zoom in" — that's the hierarchy.

This is what vector databases (Pinecone, Weaviate, Qdrant, pgvector, FAISS) implement under the hood. Interview line: "Vector DBs use ANN indexes like HNSW to get sub-linear search — that's the entire reason they exist."

4.2 (B) Scaling the TRAFFIC: your Oct 23 bottom diagram

Your drawing was:

```
Users → [ API Gateway ] → [ Load Balancer (Nginx, on AWS) ] → ☉ ☉ ☉ ☉ (many servers)
```

- **API Gateway** — the single front door. Handles authentication, rate limiting (stop one user hammering you), request routing. *Analogy: hotel reception — checks who you are before letting you to any floor.*
- **Load Balancer (Nginx)** — distributes incoming requests evenly across many identical app servers, and skips dead ones. *Analogy: the restaurant host who seats each arriving group at whichever table is free — no single waiter gets crushed.*
- **Many servers** — identical copies of your RAG app, any of them can serve any request.

4.3 Vertical vs Horizontal scaling (your Oct 24 note)

- **Vertical scaling** = make ONE server bigger (more RAM/CPU/GPU). Easy, but has a hard ceiling and one failure = total outage. *One superhuman waiter.*
- **Horizontal scaling** = add MORE servers behind the load balancer. Nearly unlimited, fault-tolerant — but now someone must manage all these machines... *Hire more normal waiters.*

Production answer: **horizontal**, always, for serving millions.

4.4 Kubernetes (your Oct 24 note)

If you have 50 server copies (containers), who:

- restarts one when it crashes?
- adds copies at peak traffic and removes them at 3 AM (auto-scaling)?
- rolls out a new version with zero downtime?

Kubernetes (K8s) — the orchestrator that does all of that automatically. You declare "I want 10 replicas of my RAG API, auto-scale to 50 under load," and K8s makes reality match.

Analogy: the restaurant manager. Waiters (containers) serve; the manager hires/fires by rush hour, replaces anyone who faints, and retrains staff without closing the restaurant.

4.5 The full picture — say this in an interview and you win

"A production RAG system at scale: requests enter via an **API Gateway** (auth, rate limits), a **load balancer** spreads them across **horizontally-scaled** app servers managed by **Kubernetes**. Each request first checks a **Redis semantic cache** — a hit returns instantly. On a miss, a **router LLM** decides: direct answer, web search, or RAG. The RAG path does **hybrid retrieval** (BM25 + vector search over an **HNSW** index) of ~100 chunks, merges lists with **RRF**, reranks the top 20 with a **cross encoder**, and sends the **top 5 chunks** to the LLM for a grounded answer — which is then cached."

Read that paragraph 5 times. It is literally your diary, Oct 19 → Oct 24, in one breath.

4.6 🖱 Hands-on (Jupyter) — feel HNSW's speed

```
!pip install faiss-cpu -q
import faiss, numpy as np, time

d, n = 384, 200_000 # 200k fake chunk vectors
data = np.random.random((n, d)).astype('float32')
query = np.random.random((1, d)).astype('float32')

# Brute force index
flat = faiss.IndexFlatL2(d); flat.add(data)
t = time.time(); flat.search(query, 5)
print(f"Brute force: {(time.time()-t)*1000:.1f} ms")

# HNSW index
hns = faiss.IndexHNSWFlat(d, 32); hns.add(data) # build takes a moment
t = time.time(); hns.search(query, 5)
print(f"HNSW: {(time.time()-t)*1000:.1f} ms")
```

Increase `n` to 1,000,000 and watch brute force crawl while HNSW stays instant.

4.7 🦿 Interview drill (Hour 4)

1. **How do you scale vector search to millions of vectors?** → ANN indexes; HNSW = layered graph, greedy search top-down, ~O(log n), tiny recall trade-off.
2. **HNSW vs brute force trade-off?** → Speed & memory vs exactness; HNSW is approximate (may occasionally miss the true #1

- neighbor) — acceptable for RAG.
3. **Horizontal vs vertical scaling?** → More machines vs bigger machine; horizontal for fault tolerance + unlimited growth; needs load balancing.
 4. **Role of load balancer / API gateway?** → LB: distribute traffic, health checks. Gateway: auth, rate limiting, routing — the single entry point.
 5. **Why Kubernetes for RAG serving?** → Auto-healing, auto-scaling with traffic, zero-downtime deploys of containerized RAG services.
 6. **Where would you add caching?** → Semantic cache (Redis) before the router; also embed-cache for repeated documents.

📁 FINAL 15 MINUTES — Master Cheat Sheet

The one flow to remember (your diary in order)

Oct 19: Basic RAG	→ chunk, embed, vector search, cosine sim
Oct 19: Hybrid Search	→ vector (meaning) + BM25 (exact words)
Oct 20: RRF	→ merge ranked lists by RANK: $\Sigma 1/(k+rank)$
Oct 20: Funnel	→ 100 chunks → RRF → top 20 → Cross Encoder → top 5
Oct 22: Cross Encoder	→ (query+doc together) → BERT → score. Accurate, slow, use last.
Oct 22: HNSW	→ $O(\log n)$ approximate vector search
Oct 23: Semantic Cache	→ "Do we need to retrieve at all?" – Redis, similarity threshold
Oct 23: Agentic RAG	→ Router LLM → web search / RAG / direct answer (LangGraph)
Oct 23: Infra	→ API Gateway → Load Balancer (Nginx) → servers (AWS)
Oct 24: Scaling	→ Horizontal > vertical; Kubernetes orchestrates

Name corrections before the interview

Your notes said	Correct term
Reciprocal <i>Range</i> Fusion	Reciprocal Rank Fusion (RRF)
Long Chain	LangChain
Long Chain Graph	LangGraph
h NSW	HNSW (Hierarchical Navigable Small World)
Kubernetes ✓	Kubernetes (pronounced koo-ber-NET-eez, "K8s")

The 3 "why" questions that separate juniors from hires

1. **Why hybrid?** Because vector misses exact tokens (ABC23) and keyword misses meaning.
2. **Why a funnel (retrieve many → rerank few)?** Because accuracy and speed are enemies: bi-encoders are fast/rough, cross-encoders slow/precise — use each where it's strong.
3. **Why cache/route before retrieving?** Because the cheapest retrieval is the one you never run.

Good luck — you've got this. 🍀